

Projektplan: Autonomes Durchspielen von Fear & Hunger

Hierarchischer LLM-/RL-Agent mit Wahrnehmung, Gedächtnis, Skill-Bibliothek und kontrollierter No-Spoiler-Wissensbasis

Erstellt für das Hobbyprojekt: Machine Learning Autonomes Spielen

Stand: 9. Mai 2026

Kernaussage	Konsequenz für das Projekt
Reines End-to-End-Reinforcement-Learning aus Pixeln ist für Fear & Hunger als Hobbyprojekt sehr wahrscheinlich unpraktisch.	Der Agent braucht hierarchische Aufgabenzerlegung, Gedächtnis, Skill-Lernen und sprachlich-symbolische Planung statt nur Tastenbelohnungen.
Der sinnvollste Aufbau ist ein hybrider Agent: LLM als Planer, Datenbank als Gedächtnis, kleine Policies/Heuristiken als Körper.	Das LLM entscheidet nicht jede Taste, sondern vergibt lokale Ziele und Belohnungen an robuste Controller.
Ein vollständiges Ending ist ein langfristiges Forschungsziel, kein realistisches erster Meilenstein.	MVP: zuverlässige Wahrnehmung, Textverständnis, Menüs, Raumgraph, erste Exploration, später Kampf- und Ressourcenstrategien.
Keine vorherige Fear-&-Hunger-Kenntnis ist mit pretrained LLMs nicht mathematisch beweisbar.	Nutze eine Wissens-Firewall: Entscheidungen dürfen nur auf beobachteten Evidenzen, generischen Gaming-Priors und verifizierten Experimenten beruhen.

Spoiler-Hinweis: Dieses Dokument behandelt Fear & Hunger fast ausschließlich technisch. Es vermeidet absichtlich konkrete Lösungswege, Ending-Bedingungen, Item-Spoiler und Guide-Wissen. Die wenigen Spielinformationen betreffen nur Engine, Oberfläche und Projektintegration.

1. Kurzfassung

Deine Projektidee ist grundsätzlich richtig ausgerichtet: Für ein Spiel mit langen Konsequenzketten, Backtracking, Trial-and-Error, textuellen Hinweisen, zufälligen Ereignissen und harten Fehlschlägen ist ein flaches RL-System mit direkter Belohnung aus Tasteneingaben keine gute Startstrategie. Der heutige Forschungsstand spricht für eine Hierarchie aus symbolisch-sprachlicher Planung, episodischem Gedächtnis, Skill-Bibliothek und lokal trainierten Controllern. Dieses Muster findet sich in Minecraft- und Game-Agent-Systemen wie VPT, MineRL/MineDojo, Voyager, Dreamer und SIMA, aber auch in älteren hierarchischen RL-Methoden wie Options, MAXQ, Option-Critic und FeUdal Networks. [5][6][7][8][9][10][20][21][22][23]

Der realistische Projektkern: Baue zuerst keinen End-to-End-Agenten, sondern ein Experimentalsystem, das das Spiel wie ein Mensch über Bildschirm, Text, Menü und Eingaben wahrnimmt. Der Agent soll Wissen nur aus eigenen Beobachtungen und Experimenten eintragen. Ein lokales LLM plant in langsamen Zyklen; ein schneller Controller führt konkrete Tastensequenzen aus; ein Lernsystem verbessert kleine Optionen wie Navigation, Interaktion, Kampfmenüs und Fluchtentscheidungen.

- **Strategische Ebene:** LLM mit ReAct-/Reflexion-ähnlichem Loop: Beobachtung, Hypothese, Plan, Aktion, Auswertung, Gedächtnisupdate.
- **Taktische Ebene:** Task-Manager, der Ziele in Skills/Options übersetzt: Raum erreichen, Text lesen, Objekt untersuchen, Menü bedienen, Kampfoption testen, Ressourcen prüfen.
- **Motorische Ebene:** Kleine Policies, Skripte oder RL-Controller, die nur lokale Aufgaben mit klaren Erfolgskriterien lernen.
- **Gedächtnis:** Episodisches Log plus semantische Knowledge Base plus Raumgraph plus Skill-Registry. Jede Spielbehauptung bekommt Evidenz, Zeit, Screenshot und Vertrauensgrad.
- **No-Spoiler-Prinzip:** Keine Wiki-/Guide-Daten, kein Lesen von Map-/Enemy-/Switch-Dateien, keine F&H-spezifischen Facts im Prompt. Das LLM darf generische Spielintuition nutzen, aber keine nicht beobachteten Details als Fakt behandeln.

2. Zieldefinition und Nicht-Ziele

Kategorie	Beschreibung
Primäres Ziel	Ein lokaler Agent soll Fear & Hunger über eine menschliche Schnittstelle autonom explorieren, Wissen aufbauen, aus Fehlschlägen lernen und über mehrere Neustarts bessere Strategien entwickeln.
Langfristiges Ziel	Mindestens ein Ending oder eine klar definierte Endbedingung erreichen, ohne externe Spoiler- oder Guide-Daten.
Erster Erfolg	Der Agent kann zuverlässig Bildschirmzustände loggen, Dialog-/Menütext lesen, Raumübergänge erkennen, einfache Ziele verfolgen und nach Tod/Reset sein Wissen sauber wiederverwenden.
Nicht-Ziel im MVP	Kein kompletter End-to-End-Sieg, kein Training eines großen Foundation Models, kein massives RL über Millionen schneller paralleler Spielinstanzen.

Kategorie	Beschreibung
Experimentelle Leitfrage	Wie weit kommt ein lokal laufender, evidence-gated LLM-Agent mit Skill-Lernen in einem harten, knowledge-basierten RPG?

3. State of Work: Was wurde in verwandten Projekten erreicht?

Die wichtigsten Resultate zeigen zwei Trends: (1) Deep RL kann Spiele meistern, wenn die Umgebung schnell, wiederholbar und stark skalierbar ist; (2) lange, offene, knowledge-lastige Spiele profitieren von menschlichen Demonstrationen, Curricula, Skill-Abstraktion, Weltmodellen oder LLM-Planern. Fear & Hunger liegt näher an der zweiten Kategorie als an Atari.

System / Feld	Was erreicht wurde	Relevanz für Fear-&Hunger-Agent
Atari: DQN und Agent57	DQN lernte aus visueller Erfahrung viele Atari-Spiele; Agent57 übertraf später menschliche Baselines auf allen 57 Atari-Spielen.	Belegt Grundfähigkeit von RL, aber Atari hat kurze Episoden, klare Scores und meist keine komplexe persistent-semantische Wissensbasis.[1][2]
AlphaStar / StarCraft II	Multi-Agent-RL mit menschlichen und agentischen Spielen erreichte Grandmaster-Level und lag über 99,8 Prozent offiziell geranker Spieler.	Zeigt, dass komplexe Spiele möglich sind, aber mit enormer Infrastruktur, Selbstspiel, Replaydaten und spezieller Umgebung.[3]
OpenAI Five / Dota 2	Großskaliges Deep RL besiegte Weltmeister; der Maßstab waren lange, teambasierte, stochastische Entscheidungen.	Nützlich als Warnung: Erfolg entstand durch massives Training, Parallelisierung und kontrollierte Schnittstellen, nicht durch Hobby-Compute.[4]
MineRL und VPT	Minecraft-Tasks wie ObtainDiamond sind wegen Exploration, Langhorizont und spärlicher Belohnung schwer. VPT nutzte Video PreTraining aus vielen Videos plus kleine gelabelte Aktionsdaten.	Für F&H wäre eine analoge Demonstrationspipeline möglich, aber Spoilerfreiheit und Datenkosten sprechen für kleinere Skill-Demos oder Proxy-Umgebungen.[5][7]
Dreamer / World Models	Dreamer lernt ein Weltmodell und plant indirekt über Vorhersagen; 2025 wurde das Sammeln von Minecraft-Diamanten aus Sparse Rewards ohne Menschendaten/Curricula berichtet.	Weltmodelle sind attraktiv, aber selbst dort sind 100M Schritte und GPU-Tage Größenordnungen über einem F&H-MVP. [6]
MineDojo und Voyager	MineDojo stellt viele sprachliche Minecraft-Aufgaben und Internetwissen bereit; Voyager kombiniert LLM, Curriculum, Skill-Bibliothek und Feedback-Loop.	Voyager ist architektonisch am nächsten: LLM plant, Skills wachsen, Erfahrungen werden reflektiert. F&H muss jedoch ohne Mineflayer-API und ohne Spielwissen auskommen.[8][9]
SIMA	DeepMind trainierte einen generalistischen Agenten, der natürliche Sprache und Bildschirmwahrnehmung in vielen 3D-Spielen mit Tastatur-/Mausaktionen verbindet.	Bestätigt die Richtung: visuelle Wahrnehmung + Sprache + Aktionen über menschliche Schnittstellen. Für ein Hobbyprojekt bleibt nur eine stark kleinere Version realistisch.[10]

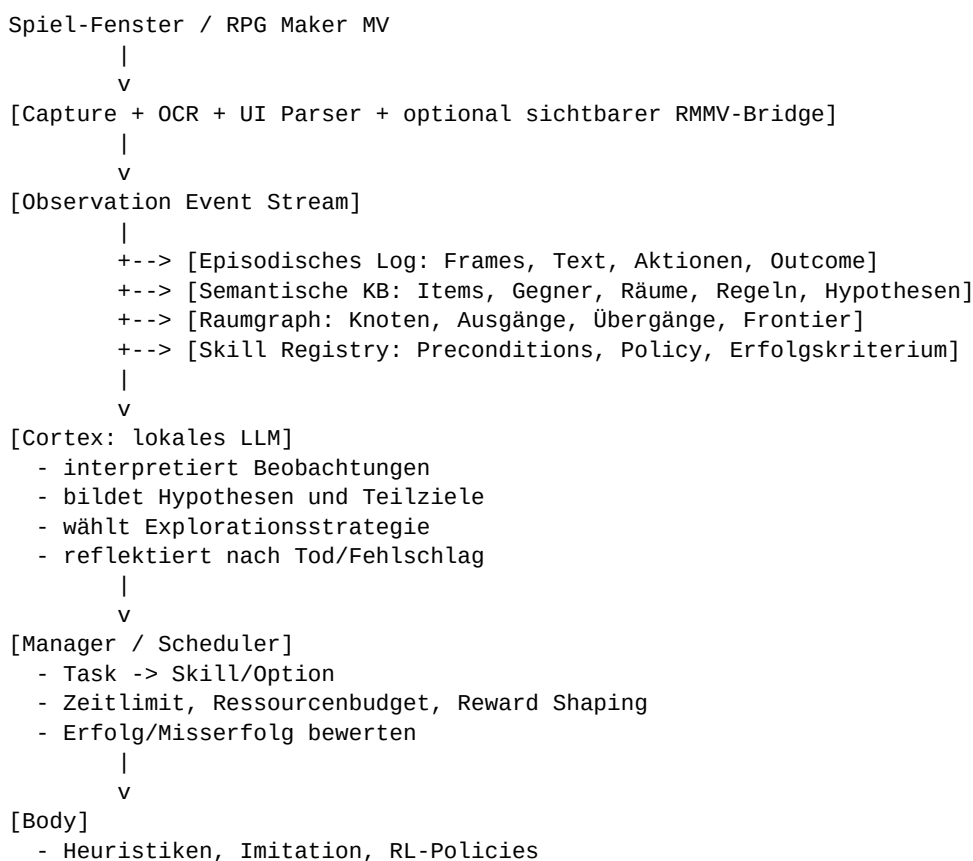
System / Feld	Was erreicht wurde	Relevanz für Fear-&Hunger-Agent
TextWorld / ALFWorld / NetHack	Forschungsumgebungen für Sprache, Exploration, Gedächtnis, Planung und lange Konsequenzen.	Sehr gute Zwischenziele: zuerst in TextWorld/ALFWorld/NLE Konzepte testen, dann auf F&H übertragen.[12][14][15]

4. Warum Fear & Hunger besonders schwer ist

- **Sparse und verspätete Belohnung:** Viele Entscheidungen haben erst viel später Konsequenzen. Ein Tastendruck kann heute gut wirken und später katastrophal sein.
- **Hohe Irreversibilität:** Tod, Ressourcenverlust, Statusveränderungen und verpasste Chancen erzeugen harte Lernsignale mit wenig unmittelbarer Erklärung.
- **Labyrinth und Backtracking:** Der Agent braucht eine eigene Karte, Raumidentität, Übergänge, Frontier-Erkundung und Rückkehrplanung.
- **Semantik statt Reflexe:** Items, Texte, Dialoge und Menüs enthalten Hinweise. Ein Agent muss Sprache und UI zuverlässig erfassen.
- **RNG und Experimentkosten:** Einzelne Beobachtungen können zufällig sein. Fakten müssen über Evidenz und Wiederholungen abgesichert werden.
- **Proprietäre, nicht parallelisierte Umgebung:** Ein Hobbysetup kann nicht tausende beschleunigte Spielinstanzen parallel fahren. Sample-Effizienz ist wichtiger als Roh-RL.

5. Architekturvorschlag

Empfohlen wird eine dreistufige Hierarchie: Cortex, Manager, Body. Das LLM ist nicht der Joystick. Es ist der reflektierende Planer, der lokale Aufgaben formuliert, Hypothesen verwaltet und nach Fehlschlägen neue Strategien erzeugt. Der Körper ist ein Satz schneller, kleiner Controller, die nur klar umrissene Optionen ausführen.



- Tastatur/Maus/Controller
- läuft schnell, LLM läuft langsam

|
v

[Input Executor -> Spiel]

Modul	Aufgabe	Empfohlene Implementierung im MVP
Perception	Bildschirm, Text, Menüs, Kampf-/Statusanzeige, Raumwechsel erfassen.	mss/OBS-Window-Capture + OpenCV + OCR; feste Auflösung; klar definierte UI-Crops; Screenshots als Evidenz.
Observation Stream	Alles, was der Agent sieht, als strukturierte Events ablegen.	JSONL-Events: timestamp, screenshot_id, text_spans, ui_state, action_before, outcome_hint.
Memory	Aus Erfahrungen eine persistente Spielwissensbasis erzeugen.	SQLite/DuckDB + Vektorindex; jedes Fact-Objekt enthält Evidenz, Konfidenz, Widersprüche.
Cortex / Planner	Großziel in lokale Ziele und Experimente zerlegen.	gpt-oss-20b oder anderes lokales Reasoning-Modell; ReAct-/Reflexion-Loop; keine direkte Tastenkontrolle.
Manager	Planner-Ausgaben in ausführbare Options übersetzen und begrenzen.	Finite-State-Machine plus Task Queue; Skill-Verträge mit Preconditions/Success/Timeout.
Body	Primitive und lokale Aktionen ausführen.	Zuerst Skripte/Heuristiken; später PPO/DQN/Imitation pro Skill; 10-30 Hz Aktionsloop.
Reward Shaper	Lokale Belohnungen aus selbst gesetzten Zielen erzeugen.	Neue Räume, gelesene Texte, Itemänderung, Überleben, Zielerreichung, Ressourcenverlust, Tod.
Experiment Controller	Neustarts, Seeds, Versionierung und Ablationen organisieren.	Run-Ordner, Configs, Checkpoints, DB-Snapshots, Replay-Auswertung.

6. Denkmodell des Agenten: Menschliche Spielweise als formale Schleife

Das Verhalten soll nicht wie ein Reflexagent wirken, sondern wie ein vorsichtiger Spieler ohne Vorwissen. Dafür reicht kein Chain-of-Thought-ähnliches Textprotokoll. Es braucht einen expliziten externen Zustand: offene Fragen, Hypothesen, getestete Aktionen, beobachtete Konsequenzen, Risikostand und strategische Prioritäten. ReAct, Reflexion, Generative Agents und Tree-of-Thoughts liefern dafür nützliche Muster: reasoning+acting, verbale Selbstkritik, dynamische Erinnerung und systematische Suche über Alternativpläne. [16][17][18][19]

Menschliche Fähigkeit	Agentische Umsetzung
Intuition: Gegenstände können nützlich sein	Generische Priors: markiere neue Items als untersuchenswert; teste sichere Verwendungen; dokumentiere Effekte.

Menschliche Fähigkeit	Agentische Umsetzung
Aufgabe zerlegen	Goal Stack: Oberziel -> Zwischenziel -> Skill -> primitive Actions.
Aus Fehlern lernen	Nach Tod/Fehlschlag: Ursache vermuten, Evidenz markieren, alternative Strategie erzeugen, Risikoregel ergänzen.
Neugier	Intrinsic Reward für neue Räume, neue UI-Texte, unbekannte Gegner, unbekannte Itemeffekte und Widersprüche.
Vorsicht	Ressourcen- und Risikoheuristiken: nicht jedes Experiment sofort wiederholen; sichere Zustände bevorzugen; Tod teuer gewichten.
Backtracking	Raumgraph mit Frontiers, Sackgassen, Kosten, Gefahrmarkierungen und bekannten Rückwegen.

```

while run_active:
    obs = perceive_screen()
    events = parse_observation(obs)
    memory.update(events)

    if manager.current_skill_finished_or_failed(events):
        result = manager.evaluate_result(events, memory)
        memory.record_skill_result(result)
        planner.reflect(result, memory.relevant_context())

    goal = planner.choose_next_goal(memory.summary(), constraints)
    skill = manager.select_skill(goal, memory)
    manager.execute(skill, body, reward_shaper)

    if death_or_reset_detected(events):
        planner.post_mortem(memory.last_run())
        reset_game_according_to_protocol()

```

7. Informationen, die aus Fear & Hunger geholt werden sollten

Für die No-Spoiler-Version gilt: Der Agent darf nur wahrnehmen, was ein Spieler im Fenster sehen könnte. Eine RPG-Maker-MV-Bridge kann später hilfreich sein, aber sie darf nur sichtbare Oberflächenzustände ausgeben. RPG Maker MV unterstützt JavaScript-Plugins; genau dadurch wäre eine instrumentierte sichtbare Schnittstelle technisch möglich. [24][25]

Datenklasse	Erlaubt in No-Spoiler-MVP	Verboten / Spoilergefährlich
Bildschirm	Screenshot, Pixel, UI-Crops, Bewegung/Übergänge.	Kein Lesen versteckter Map-Dateien, Eventnamen oder Triggerbedingungen.
Text	Aktuell sichtbarer Dialog, Menütext, Itemnamen im geöffneten Menü, Kampftext.	Keine externen Guides, Wiki-Texte, Datamining von Itembeschreibungen außerhalb beobachteter UI.

Datenklasse	Erlaubt in No-Spoiler-MVP	Verboten / Spoilergefährlich
Status	Nur sichtbare HP/Status/Inventarwerte, wenn UI geöffnet oder auf Screen sichtbar.	Keine internen Variablen, Switches, Flags oder zukünftige Konsequenzen.
Kampf	Sichtbare Gegnerdarstellung, sichtbare Optionen, beobachteter Schaden, eigene Zustände.	Keine Enemy-Datenbank, HP-Werte, Resistenzlisten, Drop-Tabellen.
Karte	Selbst gebauter Raumgraph aus beobachteten Übergängen und visuellen Signaturen.	Keine RPG-Maker-Map-JSONs, Koordinatensysteme, Event-Kommentare.
Speichern/Neustart	Agent darf Run-Ende, Tod, Save/Load-Protokoll loggen.	Keine Savegame-Inspektion nach versteckten Variablen.

8. Schnittstellenvarianten

Variante	Beschreibung	Vorteile	Nachteile / Empfehlung
A. Screen-only	Der Agent sieht nur das Spielbild und sendet Tastendrucke.	Sauberste No-Spoiler-Bedingung; menschennah; keine Engine-Modifikation.	OCR/UI schwerer; langsamere Entwicklung. Empfohlen für den ersten ernsthaften F&H-Run.
B. Sichtbare Bridge	RPG-Maker-MV-Plugin exportiert nur sichtbaren Text, aktives Menü, sichtbare Statuswerte und akzeptiert Input-Kommandos.	Deutlich stabilere Perception; weniger OCR-Fehler; trotzdem relativ fair, wenn streng beschränkt.	Plugin muss gut auditiert werden; Risiko von Hidden-State-Leak. Gut für Debug- und Vergleichsläufe.
C. White-box	Agent liest Spiel-/Map-/Save-/Enemy-Dateien und interne Variablen.	Technisch mächtig und schneller.	Zerstört No-Spoiler-Anspruch. Nur für spätere Tooling-Experimente, nicht für Kernexperiment.

Empfehlung: Starte mit A. Baue B separat als Debug-Schalter, aber deaktiviere sie in offiziellen No-Spoiler-Runs. Wenn B genutzt wird, protokolliere exakt, welche Felder exportiert werden, und verbiete im Code alle Zugriffe auf \$gameSwitches, \$gameVariables, \$gameTroop-Detaildaten und Map-Eventdaten, weil diese Hidden Knowledge enthalten können. RPG Maker MV stellt solche globalen Objekte bereit; genau deshalb muss die Bridge bewusst restriktiv sein. [25]

9. Lokale Modell- und Framework-Auswahl

9.1 LLM-Planer

gpt-oss-20b ist für dieses Projekt ein plausibler lokaler Planer. OpenAI beschreibt es als open-weight Modell für lokale oder spezialisierte Low-Latency-Anwendungen mit 21B Gesamtparametern und 3,6B aktiven Parametern; gpt-oss wurde unter Apache 2.0 veröffentlicht und für effiziente Consumer-Hardware-Deployments beworben. [26][27]

Modelltyp	Rolle	Einschätzung für 6900 XT / 32 GB RAM
gpt-oss-20b quantisiert	Hauptplaner, Reflexion, Task-Zerlegung, Tool-Aufrufe über lokale API.	Gute erste Wahl. 16 GB VRAM sind realistisch knapp, aber quantisiert/mit Offload nutzbar. Kontext nicht unnötig groß wählen.
Qwen3-14B / Qwen3-30B-A3B quantisiert	Alternative Planer; MoE-Variante interessant wegen weniger aktiver Parameter.	30B-A3B kann trotz MoE speicherintensiv sein; 14B oder 20B-Klasse ist pragmatischer.[36]
Qwen3-VL 8B oder Qwen2.5-VL 7B	Periodische Screenshot-Interpretation, UI-Klassifikation, visuelle Plausibilitätsprüfung.	Nur sequenziell mit LLM betreiben oder CPU/GPU-Offload, sonst VRAM-Konflikt. OCR zuerst nutzen.[35]
Kleines Embedding-Modell	Ähnliche Erfahrungen, Dialoge und Raum-/Itemnotizen finden.	CPU reicht oft. Muss nicht groß sein; wichtiger sind saubere Evidenz-IDs.

9.2 Inferenz-Stack

Komponente	Empfehlung	Begründung
LLM Runtime	Ollama, llama.cpp, LM Studio oder vLLM	llama.cpp läuft auf CPU/GPU und unterstützt Quantisierung; ROCm-Dokumentation beschreibt llama.cpp als LLM-Inferenzframework mit Quantisierungsoptionen. vLLM ist nützlich, wenn du eine OpenAI-kompatible lokale Server-API willst.[29][37]
GPU-Unterstützung	ROCm/HIP oder Vulkan-Backend testen	AMD führt RX 6900 XT in der HIP-SDK-Supporttabelle mit RDNA2/gfx1030 und Runtime/HIP-SDK-Unterstützung auf.[28]
OCR	PaddleOCR / PP-OCRv5	PP-OCRv5 unterstützt u. a. Englisch und komplexe Szenarien; für RPG-Menüs ist OCR-Fehlerbehandlung trotzdem Pflicht.[34]
RL API	Gymnasium-kompatible Env-Schicht	Gymnasium ist der gepflegte Gym-Nachfolger und Standard-Interface für RL-Probleme.[30]
RL Library	Stable-Baselines3 für MVP, CleanRL für nachvollziehbare Experimente, RLlib nur bei Skalierung	SB3 liefert verlässliche PyTorch-Implementierungen; CleanRL ist gut lesbar und experimentell; RLlib ist auf skalierbare Workloads ausgelegt.[31][32][33]

9.3 Hardware-Bewertung

Komponente	Bewertung
Ryzen 7 9800X3D	Sehr stark für Spiel, Python-Orchestrierung, CPU-Offload und lokale Simulationen. Der Engpass wird eher GPU-VRAM, RAM und Sample-Throughput sein.

Komponente	Bewertung
32 GB DDR5	Für Spiel + LLM + OCR + Logs ausreichend für MVP, aber knapp für parallele VLM/LLM/RL-Training. 64 GB wäre die erste sinnvolle Aufrüstung.
Radeon RX 6900 XT, 16 GB VRAM	Gut für quantisierte 7B- bis 20B-Klasse und kleine RL-Modelle. Kein komfortables Setup für gleichzeitiges großes VLM + 20B-LLM + Training.
Lokales Setup ohne API	Kostenkontrolle ist gut, aber Qualität/Latency sind begrenzt. Plane langsame Denkzyklen: LLM nur bei Ereignissen oder alle 1-10 Sekunden, nicht für jede Taste.
Parallelisierung	Fear & Hunger selbst ist wahrscheinlich der Sample-Engpass. Ohne beschleunigte parallele Instanzen muss das System sample-effizient und stark heuristisch sein.

10. Reward-Design und Skill-Lernen

Die zentrale Idee ist: Das große Ziel wird nicht direkt belohnt. Der Planer formuliert lokale Tasks; für jeden Task gibt es Erfolgskriterien, Metriken und ein kleines Reward-Modell. Hierarchische RL-Forschung nennt solche temporär erweiterten Aktionen häufig Options; sie können Planung und Lernen über lange Horizonte deutlich erleichtern. [20][22][23]

Reward-Klasse	Beispiel	Zweck
Extrinsischer Fortschritt	Neuer Raum stabil erreicht; neuer Dialog gelesen; neues Item sichtbar im Inventar; Kampf überlebt; Zielraum erreicht.	Lokale Aufgaben belohnen, ohne das Ending kennen zu müssen.
Intrinsische Neugier	Unbekannter Ausgang, unbekannter Menüpunkt, nicht getestetes Item, neue visuelle Raum-Signatur.	Exploration antreiben, wenn externe Belohnung fehlt.
Informationsgewinn	Hypothese wurde bestätigt/widerlegt; Itemeffekt dokumentiert; Gegnerverhalten beobachtet.	Trial-and-Error als Wissensaufbau formalisieren.
Risikokosten	Tod, kritischer HP-Verlust, Ressourcenverbrauch, Endlosschleife, unproduktive Wiederholung.	Blindes Experimentieren begrenzen.
Skill-Erfolg	Skill `open_menu_inventory` erreicht Zustand X in unter N Schritten; `return_to_known_room` findet Rückweg.	Controller trainieren, ohne lange Story-Abhängigkeit.

Skill / Option	Success Detector	Start mit
read_visible_text	OCR liefert stabilen Text oder Bridge liefert aktuelles Textfenster; Text wurde gespeichert.	Regel/Parser.

Skill / Option	Success Detector	Start mit
navigate_to_exit	Raumwechsel oder visuelle Transition erkannt.	Heuristik + später RL/Imitation.
build_room_graph	Neuer/alter Raumknoten korrekt zugeordnet, Ausgangskante gespeichert.	Algorithmisch.
open_and_parse_inventory	Menüzustand erkannt, Itemliste extrahiert.	Makro + OCR.
interact_with_frontier_object	Interaktionsversuch erzeugt neues Event, Text, Menü oder keine Änderung; Ergebnis geloggt.	Heuristik.
combat_basic_survival	Kampfstadium verlassen und Spielfigur lebt oder sauberer Tod/Outcome geloggt.	Zunächst Regeln; später lokale Policy.
retreat_or_reset_decision	Risiko-/Ressourcenregel löst Flucht/Reset aus.	LLM/Manager-Regel, kein RL zuerst.

11. Wissensbasis und No-Spoiler-Firewall

Wichtig: Ein pretrained LLM kann nicht garantiert ohne Fear-&Hunger-Vorwissen sein. Es könnte während des Trainings Fan-Wikis, Diskussionen oder Reviews gesehen haben. Deshalb ist die sauberste wissenschaftliche Formulierung nicht 'das Modell weiß nichts', sondern: 'Der Agent darf in Entscheidungen nur evidenzgeprüfte Fakten aus eigenen Beobachtungen verwenden; generische Spielintuition ist erlaubt, aber unbewiesene F&H-Details werden blockiert.'

Mechanismus	Umsetzung
Evidenzpflicht	Jede game-spezifische Behauptung braucht evidence_id: Screenshot, OCR-Text, Run, Zeitpunkt, Aktion, Outcome.
Faktentypen	fact, hypothesis, rule, contradiction, todo_experiment. Nur fact/rule mit ausreichender Evidenz dürfen strategisch stark gewichtet werden.
Prompt-Gate	Systemprompt: 'Nutze keine externen Fakten über Fear & Hunger. Wenn du etwas nicht im Beobachtungslog findest, markiere es als Hypothese.'
Retriever-Gate	Das LLM bekommt nur Memory-Summaries aus der eigenen DB, keine Websuche, keine Guides, keine Rohdaten aus Spieldateien.
Canary-Test	Vor offiziellem Run das Modell nach F&H-Details fragen und protokollieren, ob Vorwissen vorhanden ist. Danach trotzdem Evidence-Gate nutzen.
Modellvergleich	Optional: kleines weniger wissendes Modell vs. stärkeres Modell mit striktem Gate vergleichen.

Mechanismus	Umsetzung
Audit	Regelmäßige Prüfung von Planner-Entscheidungen: wurde ein Fakt ohne evidence_id benutzt? Dann Run als kontaminiert markieren.

KnowledgeFact schema

```
{
  "fact_id": "f_000123",
  "claim": "Item X appeared in inventory after action Y",
  "status": "hypothesis|observed_fact|validated_rule|contradicted",
  "confidence": 0.0-1.0,
  "evidence_ids": ["ev_2026_05_09_run03_001244"],
  "first_seen": "timestamp",
  "last_tested": "timestamp",
  "contradictions": [],
  "next_experiment": "optional"
}
```

Planner rule:

- Use game-specific claims only if they appear in KnowledgeFact with evidence.
- Generic priors are allowed only as priors, never as factual F&H knowledge.
- When uncertain, propose a low-risk experiment instead of inventing a rule.

12. Daten- und Log-Architektur

Artefakt	Speicherformat	Zweck
events.jsonl	JSONL pro Run	Chronologische Beobachtungen, Aktionen, Parserausgaben und Outcome-Hints.
screenshots/	PNG/JPEG mit Hash	Evidenz, Debugging, später VLM-/OCR-Reprocessing.
video/	Optional MP4 pro Run	Human Review und Fehleranalyse.
memory.sqlite	SQLite oder DuckDB	Facts, Hypothesen, Raumgraph, Skill-Ergebnisse, Inventarverläufe.
vectors/	FAISS/Chroma oder einfacher Embedding-Index	Ähnliche Texte/Erfahrungen schnell abrufen.
checkpoints/	Policy-Weights und Skill-Versionen	Lernfortschritt pro Option.
configs/	YAML	No-Spoiler-Modus, Screen-Resolution, Modell, Skill-Timeouts, Random Seeds.
evals/	CSV/JSON	Metriken über Runs und Ablations.

ObservationEvent schema

```
{
  "event_id": "ev_...",
  "run_id": "run_0007",
  "t": 183.42,
  "screenshot_id": "ss_000412.png",
  "ui_state": "field|dialogue|menu|combat|loading|death|unknown",
  "visible_text": [{"text": "...", "bbox": [x1,y1,x2,y2], "confidence": 0.91}],
  "visual_signature": "roomhash_...",
}
```

```

"agent_action": {"skill": "navigate_to_exit", "keys": ["up", "enter"]},
"outcome_hint": "new_text|room_transition|inventory_change|damage|death|no_change"
}

```

SkillResult schema

```

{
  "skill": "navigate_to_exit",
  "task_goal": "try north exit from current room",
  "success": true,
  "steps": 47,
  "reward": 1.2,
  "new_evidence": ["ev_..."],
  "failure_reason": null
}

```

13. Entwicklungsplan

Phase	Ziel	Konkrete Deliverables	Abbruch-/Erfolgskriterium
0. Projektbasis	Saubere lokale Entwicklungsumgebung.	Repo, venv/conda, Configsystem, Logging, feste Spielauflösung, Run-Ordner.	Ein Testskript kann Spielscreen erfassen und eine Taste senden.
1. Perception-Harness	Stabile Beobachtung ohne Intelligenz.	Window capture, Screenshot-IDs, OCR-Crops, UI-State-Klassifikator, Videolog.	Dialog-/Menütext wird reproduzierbar erkannt; Actions erscheinen im Log.
2. Manuelle/heuristische Skills	Körpergrundlagen.	Tastenmakros für Menü, Interaktion, Bewegung; Erfolgserkennung; Timeout-Handling.	Agent kann 10-20 Minuten ohne Crash loggen und einfache Menüs bedienen.
3. Gedächtnis	Aus Runs wird Wissen.	SQLite-Schema, Facts/Hypothesen, Raumgraph, Evidenzlinks, Memory-Summaries.	Nach Reset bleiben bekannte Räume/Texte/Hypothesen abrufbar.
4. LLM-Cortex	Planer trifft lokale Entscheidungen.	Lokaler LLM-Client, Prompts, ReAct/Reflexion, Task Queue, Evidence-Gate.	LLM wählt sinnvolle nächste Experimente statt Endlosschleifen.
5. Proxy-Training	RL nicht direkt in F&H starten.	Mini-Gridworld/RPG-Maker-Testprojekt/TextWorld-artige Proxy-Tasks für Navigation/Menüs.	Policies lernen lokale Skills in beschleunigbaren Umgebungen.
6. Online-Skill-Lernen	Kleine Policies in F&H feinjustieren.	Gymnasium-Env wrapper, SB3/CleanRL Experimente, Replay Buffer pro Skill.	Mindestens ein Skill verbessert sich messbar gegenüber Heuristik.
7. Langläufe und Ablationen	Projekt wissenschaftlich auswerten.	Run-Metriken, Baselines, no-memory/with-memory, LLM-only/RL-only/Hybrid-Vergleich.	Agent erreicht mehr Räume/Events/Überlebenszeit als Baselines.
8. Ending-Versuch	Langfristiger autonomer Fortschritt.	Stabile Reset-Strategie, Strategie-Kompression, Run-Auswahl, Risiko-Management.	Nur starten, wenn Perception, Memory und Basis-Skills stabil sind.

14. MVP: Realistisches erstes Setup

- **Betriebssystem:** Windows ist für Steam/Game/Automation bequem; Linux/Proton ist für ML/ROCm oft sauberer. Für den MVP reicht Windows, solange Capture/Input stabil sind. Bei ROCm-Problemen: Linux-Partition oder WSL2 getrennt testen.
- **Spiel:** Windowed mode, feste Auflösung, keine Overlays, keine Skalierungsänderung. Alle Runs identisch konfigurieren.
- **Perception:** Screen capture 5-15 FPS; UI-Crops; PaddleOCR; optional periodische VLM-Prüfung nur bei Unsicherheit.
- **LLM:** gpt-oss-20b quantisiert, lokale Server-API; Kontext knapp halten; nur Memory-Summaries einfügen.
- **Controller:** Python-Action-Executor mit Rate-Limits, Debounce, Focus-Check, Not-Aus.
- **Datenbank:** SQLite als Single-Source-of-Truth; jede Planner-Ausgabe und jede Hypothese loggen.
- **RL:** Zunächst gar nicht oder nur in Proxy-Tasks. In F&H erst, wenn Success-Detectors für Skills zuverlässig sind.

MVP stack

- Python 3.11/3.12
- mss or OBS virtual capture + OpenCV
- PaddleOCR for text
- local LLM server: gpt-oss-20b quantized via Ollama/llama.cpp/LM Studio/vLLM where supported
- SQLite/DuckDB + JSONL event logs
- Gymnasium wrapper for skill environments
- Stable-Baselines3 or CleanRL for later option policies
- Optional: Qwen3-VL 8B for screenshot adjudication, not every frame

15. Projektstruktur für Codex

```
fh-agent/
  README.md
  pyproject.toml
  configs/
    default.yaml
    no_spoiler.yaml
    models.yaml
  env/
    capture.py
    input_executor.py
    focus_guard.py
    gym_fh.py
    rmmv_visible_bridge.js      # optional, disabled for strict runs
  perception/
    ocr.py
    ui_state.py
    crop_registry.py
    visual_hash.py
    text_parser.py
  memory/
    schema.sql
    db.py
    evidence.py
    kb.py
    room_graph.py
    vector_index.py
  planner/
    llm_client.py
    prompts/
      system_no_spoiler.md
      react_step.md
      reflection.md
    task_manager.py
```

```

goal_stack.py
reflection.py
skills/
  registry.py
  base.py
  movement.py
  menu.py
  dialogue.py
  exploration.py
  combat.py
  rl/
    policies.py
    train_skill.py
training/
  proxy_envs/
    grid_rpg.py
    menu_env.py
  sb3/
  cleanrl/
evals/
  metrics.py
  ablations.py
  reports.py
runs/
  .gitkeep
tests/
  test_memory.py
  test_ui_state.py
  test_skill_contracts.py

```

Erste Codex-Tickets	Akzeptanzkriterium
Implementiere `capture.py` mit Window-Find, Screenshot, FPS-Limit und File-Output.	Ein Befehl speichert 100 Frames mit Zeitstempeln und Hashes.
Implementiere `input_executor.py` mit sicheren Key-Sequenzen und Not-Aus.	Testfenster erhält definierte Key-Sequenz; Not-Aus bricht ab.
Implementiere `events.jsonl` Logging.	Jede Aktion und jeder Frame bekommt run_id, event_id, timestamp.
Implementiere OCR-Crop-Registry.	Crops können per YAML definiert und debug-visuell gespeichert werden.
Implementiere `ui_state.py` als heuristischen Klassifikator.	Unterscheidet Feld/Dialog/Menü/Kampf/Death/Unknown auf Testframes.
Implementiere SQLite-Schema für Evidence, Facts, Skills, Rooms.	CRUD-Tests bestehen; Facts brauchen mindestens eine evidence_id.
Implementiere lokalen LLM-Client mit OpenAI-kompatibler API.	Kann Modell lokal ansprechen, aber nur Memory-Summary statt Rohlog senden.
Implementiere Task/Skill-Vertrag.	Skill hat preconditions, timeout, success_detector, reward_function, result.
Implementiere ersten Skill `read_visible_text`.	OCR-Ergebnis wird in Evidence und optional Fact gespeichert.
Implementiere `post_mortem` Reflection.	Nach Death-Event entsteht ein strukturierter Lernbericht mit Hypothesen und nächsten Experimenten.

16. Evaluationsplan

Metrik	Warum wichtig
Zeit bis Tod / Run-Länge	Misst Überlebensfähigkeit, aber allein nicht ausreichend.
Anzahl eindeutiger Raumknoten	Misst Exploration und Map-Aufbau.
Neue Texte/Facts pro Stunde	Misst semantischen Fortschritt und Wissensgewinn.
Skill-Erfolgsrate	Zeigt, ob Körper-Controller wirklich besser werden.
Backtracking-Erfolg	Kritisch für labyrinthartige Spiele.
Ressourcenverlust pro Fortschritt	Misst Risiko- und Ressourcenmanagement.
Loop-Rate	Wie oft wiederholt der Agent nutzlose Aktionen oder Räume?
Evidence-Violations	Wie oft benutzt der Planer nicht belegte F&H-Fakten?

Baseline	Beschreibung
Random	Zufällige Tasten mit minimaler Safety. Erwartet schlecht, aber als Untergrenze nützlich.
Heuristik-only	Exploration, Menü und Interaktion ohne LLM.
LLM + Heuristiken	Planer entscheidet, Körper bleibt regelbasiert.
RL-only pro Skill	Keine LLM-Planung, nur lokale Policy-Ziele.
Hybrid	Vollsystem mit Memory, Planer, Manager und Body.
Ablation ohne Memory	Zeigt, ob persistentes Wissen wirklich wirkt.

17. Risiken und Gegenmaßnahmen

Risiko	Gegenmaßnahme
OCR-Fehler erzeugen falsche Fakten	Jede Texterkennung mit Confidence, Screenshot und Reprocessing speichern; unsichere Facts als Hypothese.
LLM halluziniert Spielwissen	Evidence-Gate, promptbasierte Verbote, Fact-Audits, Canary-Test, Entscheidungen mit evidence_id verlangen.
Controller verliert Fokus oder sendet Tasten ans falsche Fenster	Focus Guard, Window Handle Check, Not-Aus, keine globalen Hotkeys ohne Prüfung.
Endlosschleifen	Loop-Detektor: gleiche Raum-/Action-Sequenz ohne neuen Event -> Planner muss Strategie wechseln.
RL lernt auf instabile Rewards	Erst Success-Detectors stabilisieren; RL pro Skill isolieren; Proxy-Umgebungen nutzen.

Risiko	Gegenmaßnahme
Hardware zu knapp	LLM und VLM sequenziell statt parallel; Quantisierung; Kontext kürzen; 64 GB RAM erwägen.
No-Spoiler-Kontamination	Keine Spieldateien, keine Guides, keine F&H-spezifischen Quellen in Prompts; Run als kontaminiert markieren, wenn Hidden-State genutzt wurde.
Rechtliche/ToS-Fragen	Nur lokal/single-player; EULA prüfen; keine geschützten Assets veröffentlichen; Logs ohne Spielassets teilen oder schwärzen.

18. Was du nicht zuerst tun solltest

- Kein monatelanges End-to-End-RL direkt aus Pixeln starten.
- Keine kompletten F&H-Dateien parsen, wenn das Ziel No-Spoiler-Lernen ist.
- Kein LLM pro Frame aufrufen; es ist zu langsam und zu teuer in Latenz.
- Keine Kampfstrategie auf Basis einzelner zufälliger Beobachtungen als Fakt speichern.
- Keine sehr große Modellkombination gleichzeitig in 16 GB VRAM erzwingen.
- Kein vollständiges Ending als erster Milestone definieren; erst Wahrnehmung, Memory und Skills stabilisieren.

19. Konkrete Architekturentscheidung für dein Setup

Entscheidung	Empfohlene Wahl
Primäre Agentenform	Hierarchischer LLM-Agent mit Skill-Manager und lokalen Controllern.
Perception im offiziellen Run	Screen-only + OCR. Optional VLM nur zur Bildinterpretation, nicht mit externem Wissen.
Debug-Variante	Sichtbare RMMV-Bridge nur für Text/UI, strikt ohne Hidden-State.
Planer	gpt-oss-20b quantisiert als lokaler Reasoning-Planer.
Visionsmodell	Erst nicht nötig; später Qwen3-VL 8B oder Qwen2.5-VL 7B für unklare Screenshots.
RL	Zuerst Proxy-Tasks und einzelne Skills, nicht Gesamtspiel.
Speicher	SQLite + JSONL + Screenshots; später Vektorindex.
Hardware-Ziel	MVP mit vorhandener Hardware machbar; 64 GB RAM als sinnvolle optionale Aufrüstung.
Erfolgsdefinition nach 4-8 Wochen	Agent kann 30-60 Minuten autonom explorieren, neue Fakten speichern, nach Tod einen besseren Plan formulieren und wiederholte Schleifen vermeiden.

20. Empfohlene Lese- und Videoquellen

Quelle	Warum ansehen/lesen
Voyager Paper/Projekt/Demo	Beste Blaupause für LLM + Skill-Bibliothek + automatisches Curriculum; trotz Minecraft-API sehr relevant.[9][38]
OpenAI VPT	Zeigt, wie Video-/Demonstrationsdaten primitive Aktionen aus natürlichem Interface lernen können.[5]
MineRL / MineDojo	Gute Beispiele für lange offene Spiele, Sprache, Aufgaben und menschliche Priors.[7][8]
Dreamer / World Models	Wichtiges Paradigma für sample-effizientere RL-Agenten mit interner Vorhersage.[6]
SIMA / SIMA 2	Aktueller Stand zu Sprachinstruktion + Spielbild + Tastatur/Maus in vielen Spielen; Video als Architekturintuition.[10][39]
ReAct, Reflexion, Generative Agents, Tree of Thoughts	Bausteine für Planer, Gedächtnis, Trial-and-Error-Reflexion und Alternativensuche.[16][17][18][19]
Options, MAXQ, Option-Critic, FeUdal Networks	Klassische und moderne Theorie hinter Skill-/Subtask-Hierarchien.[20][21][22][23]
NetHack, TextWorld, ALFWorld	Gute Benchmarks für Sprache, Gedächtnis und Exploration; als Zwischenübungen sehr geeignet.[12][14][15]
AI plays Fear & Hunger Videos	Nur als Unterhaltung und Inspiration. Nicht als technische Referenz und nicht in die No-Spoiler-Wissensbasis einspeisen.[40]

21. Startprompt für den lokalen Planer

System prompt sketch for official no-spoiler runs:

You are the strategic planner of an autonomous game-playing agent.

You do not know Fear & Hunger facts unless they appear in the provided observation memory.

Use generic video-game reasoning only as priors, not as factual claims.

Every game-specific claim must cite an evidence_id from memory.

If evidence is missing, mark it as a hypothesis and propose a safe experiment.

Never request web access, guides, wiki pages, datamined files, map JSON, enemy databases, switch variables, or savegame internals.

Output exactly:

1. current_belief_state
2. open_questions
3. next_goal
4. selected_skill
5. success_condition
6. risk_limit
7. memory_updates_requested

22. Fazit

Das Projekt ist als Forschungshobby sehr gut geeignet, wenn du den Anspruch richtig staffelst. Der wissenschaftlich interessante Teil ist nicht `ein LLM spielt ein Spiel`, sondern ein kontrollierter, evidenzbasierter Lernagent, der aus eigener Erfahrung eine Spieltheorie aufbaut und seine lokalen

Fähigkeiten verbessert. Der wichtigste erste Meilenstein ist daher nicht ein Ending, sondern eine robuste Schleife aus Wahrnehmung, Aktion, Gedächtnis, Hypothese, Experiment und Reflexion.

Meine klare Empfehlung: Baue zuerst das Screen-only-Harness mit OCR, Logging und Skill-Verträgen. Danach integrierst du gpt-oss-20b als langsamen Planer. Erst wenn die Skill-Erfolgserkennung stabil ist, ergänzt du RL. So vermeidest du, dass ein großes, interessantes Projekt an instabiler Perception oder unklaren Belohnungen scheitert.

Quellenverzeichnis

*Die Quellen wurden für Forschungsstand, technische Machbarkeit und Framework-Auswahl herangezogen.
Stand der Recherche: 09.05.2026.*

- [1] Mnih et al. / DeepMind: Human-level control through deep reinforcement learning (Nature, 2015).
<https://www.nature.com/articles/nature14236>
- [2] Badia et al.: Agent57: Outperforming the Atari Human Benchmark (arXiv, 2020). <https://arxiv.org/abs/2003.13350>
- [3] Vinyals et al.: Grandmaster level in StarCraft II using multi-agent reinforcement learning (Nature, 2019).
<https://www.nature.com/articles/s41586-019-1724-z>
- [4] Berner et al.: Dota 2 with Large Scale Deep Reinforcement Learning (arXiv, 2019). <https://arxiv.org/abs/1912.06680>
- [5] OpenAI: Learning to play Minecraft with Video PreTraining (VPT, 2022). <https://openai.com/index/vpt/>
- [6] Hafner et al.: Mastering diverse control tasks through world models / Dreamer (Nature, 2025).
<https://www.nature.com/articles/s41586-025-08744-2>
- [7] Guss et al.: The MineRL 2019 Competition on Sample Efficient RL using Human Priors.
<https://arxiv.org/abs/1904.10079>
- [8] MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. <https://minedojo.org/>
- [9] Wang et al.: Voyager: An Open-Ended Embodied Agent with Large Language Models.
<https://arxiv.org/abs/2305.16291>
- [10] Google DeepMind: SIMA, a generalist AI agent for 3D virtual environments. <https://deepmind.google/blog/sima-generalist-ai-agent-for-3d-virtual-environments/>
- [11] Reed et al.: A Generalist Agent / Gato. <https://arxiv.org/abs/2205.06175>
- [12] Küttler et al.: The NetHack Learning Environment. <https://arxiv.org/abs/2006.13760>
- [13] Hafner: Benchmarking the Spectrum of Agent Capabilities / Crafter. <https://arxiv.org/abs/2109.06780>
- [14] Côté et al.: TextWorld: A Learning Environment for Text-based Games. <https://arxiv.org/abs/1806.11532>
- [15] Shridhar et al.: ALFWorld: Aligning Text and Embodied Environments. <https://arxiv.org/abs/2010.03768>
- [16] Yao et al.: ReAct: Synergizing Reasoning and Acting in Language Models. <https://arxiv.org/abs/2210.03629>
- [17] Shinn et al.: Reflexion: Language Agents with Verbal Reinforcement Learning. <https://arxiv.org/abs/2303.11366>
- [18] Park et al.: Generative Agents: Interactive Simulacra of Human Behavior. <https://arxiv.org/abs/2304.03442>
- [19] Yao et al.: Tree of Thoughts: Deliberate Problem Solving with Large Language Models.
<https://arxiv.org/abs/2305.10601>
- [20] Sutton, Precup, Singh: Between MDPs and semi-MDPs: a framework for temporal abstraction in RL. <https://www-anw.cs.umass.edu/~barto/courses/cs687/Sutton-Precup-Singh-AIJ99.pdf>
- [21] Dietterich: Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition.
<https://arxiv.org/abs/cs/9905014>
- [22] Bacon, Harb, Precup: The Option-Critic Architecture. <https://arxiv.org/abs/1609.05140>
- [23] Vezhnevets et al.: FeUdal Networks for Hierarchical Reinforcement Learning. <https://arxiv.org/abs/1703.01161>
- [24] Fear & Hunger - technische Übersicht / Engine-Hinweise. https://en.wikipedia.org/wiki/Fear_%26_Hunger
- [25] RPG Maker MV: Plugin Specifications. https://rpgmakerofficial.com/product/MV_Help/page/01_11_03.html
- [26] OpenAI: Introducing gpt-oss. <https://openai.com/index/introducing-gpt-oss/>
- [27] OpenAI model docs: gpt-oss-20b. <https://developers.openai.com/api/docs/models/gpt-oss-20b>
- [28] AMD ROCm / HIP SDK for Windows: System requirements. <https://rocm.docs.amd.com/projects/install-on-windows/en/latest/reference/system-requirements.html>
- [29] AMD ROCm docs: llama.cpp on ROCm.
<https://rocm.docs.amd.com/projects/llama-cpp/en/docs-25.08/index.html>
- [30] Gymnasium Documentation. <https://gymnasium.farama.org/index.html>
- [31] Stable-Baselines3 Documentation. <https://stable-baselines3.readthedocs.io/en/master/>
- [32] Ray RLlib Documentation. <https://docs.ray.io/en/latest/rllib/index.html>
- [33] CleanRL Documentation. <https://docs.cleanrl.dev/>

[34] PaddleOCR / PP-OCRv5 Documentation.

<https://www.paddleocr.ai/main/en/version3.x/algorithm/PP-OCRv5/PP-OCRv5.html>

[35] Qwen3-VL Technical Report. <https://arxiv.org/abs/2511.21631>

[36] Qwen3 Technical Report. <https://arxiv.org/abs/2505.09388>

[37] vLLM Documentation. <https://docs.vllm.ai/>

[38] Voyager demo video. https://www.youtube.com/watch?v=BU3w_AbCEbA

[39] Google DeepMind SIMA 2 video. <https://www.youtube.com/watch?v=Zphax4f6Rls>

[40] Entertainment/non-technical: AI plays Fear & Hunger video. <https://www.youtube.com/watch?v=ubYxQiSHdwE>